A Bug in GitHub's Rate Limiter: The Wobbling Reset

Source: Arpit Bhayani Dissection • GitHub Infrastructure
Focus: Redis TTL Math, Clock Drift & Distributed Time

1. The Problem Space: Rate Limiting Contracts

GitHub enforces rate limits across billions of API calls (e.g. 5,000 requests/hour per authenticated token) using Redis as an in-memory counter store. Every API response includes metadata headers:

- `X-RateLimit-Limit`: Total quota (5,000).
- `X-RateLimit-Remaining`: Remaining requests before throttling.
- `X-RateLimit-Reset`: Unix epoch timestamp marking the exact second when the 1-hour window resets.

Developer SDKs and automated retry queues rely on `X-RateLimit-Reset` to schedule backoffs: if throttled, clients sleep until the reset timestamp arrives.

2. The Anomaly: The "Wobbling" Reset Timestamp

Developers observed that when sending rapid, concurrent requests within the exact same second, `X-RateLimit-Reset` was not stable. It **wobbled back and forth** by one second across consecutive calls:

```
Client Request 1 (t = 12:00:00.120) ---> 200 OK | X-RateLimit-Reset: 1724064530
Client Request 2 (t = 12:00:00.150) ---> 200 OK | X-RateLimit-Reset: 1724064531  (+1s jump!)
Client Request 3 (t = 12:00:00.180) ---> 200 OK | X-RateLimit-Reset: 1724064530  (-1s jump back!)

Impact: Automated client SDKs computing sleep intervals were thrown into erratic retry loops.
```

3. Root Cause: Dynamic TTL Arithmetic Flaws

The bug was caused by how GitHub originally computed the reset timestamp: rather than storing an immutable expiration timestamp upon window creation, application servers calculated it **dynamically on every request**:

> **The Flawed Dynamic Formula:** $$ ext{ResetTimestamp} = ext{Time.now} + ext{redis.ttl}(ext{key})$$ This combined two separate systems and two separate clocks: the application server host clock (`Time.now`) and the Redis server's internal TTL countdown timer.

4. Why the Dynamic Arithmetic Wobbled

The calculation collapsed under three compounding distributed phenomena:

```
+-------------------------------------------------------------------------------------------------------+
|                                    THE DISTRIBUTED TIME TRAP                                          |
+-------------------------------------------------------------------------------------------------------+
  1. Integer Truncation in Redis TTL:
     - Redis returns TTL as integer seconds (e.g., 3599s).
     - Both 3599.9s and 3599.1s return the exact same integer: 3599.

  2. Network Latency Jitter:
     - Evaluating Time.now across a 5ms network hop to Redis caused requests crossing integer
       second boundaries to round up or down inconsistently.

  3. Clock Drift Across App Server Fleet:
     - GitHub routes traffic across hundreds of application server nodes.
     - Despite NTP synchronization, sub-millisecond clock drift between nodes caused consecutive
       requests to evaluate different Time.now base values.
```

5. The Fix: Absolute Timestamp Persistence

The solution rested on a foundational distributed systems principle: **Never compute fixed truths dynamically from moving parts.**

- **Compute Once at Creation:** When a rate-limit window is first initialized, the application computes the absolute expiration timestamp once: $$ ext{ResetTimestamp} = ext{WindowStartTime} + 3600$$
- **Persist in Redis:** The absolute timestamp is saved permanently inside Redis alongside the request counter.
- **Immutable Reads:** Every subsequent request simply reads the stored `reset_at` field. It never changes throughout the 1-hour window.

> **The Zero-Wobble Invariant:** Because the timestamp is static and persisted in one place, all application servers return the exact same integer timestamp to clients, completely eliminating the wobble.

Atomic Lua Implementation & Systems Lessons

6. Production Implementation: Atomic Redis Lua Script

To prevent race conditions between window initialization, incrementing counts, and reading timestamps, GitHub moved the logic into an atomic **Redis Lua Script**:

```
-- rate_limiter.lua (Atomic Window Initialization & Increment)
local key   = KEYS[1]
local limit = tonumber(ARGV[1])
local win   = tonumber(ARGV[2])
local now   = tonumber(ARGV[3])

local data = redis.call("HMGET", key, "count", "reset_at")
local count = tonumber(data[1])
local reset_at = tonumber(data[2])

if not count then
    count = 1
    reset_at = now + win
    redis.call("HMSET", key, "count", count, "reset_at", reset_at)
    redis.call("EXPIRE", key, win)
else
    count = redis.call("HINCRBY", key, "count", 1)
end

return { count <= limit and 1 or 0, count, math.max(0, limit - count), reset_at }
```

7. First-Principles System Design Takeaways

Architecture Vector	Flawed GitHub Implementation	Production Engineering Standard
Source of Truth	Dynamic calculation: `Time.now + redis.ttl()`	Persisted absolute timestamp: `reset_at`
Clock Coordination	Coupled app server wall clock with Redis TTL clock	Decoupled; uses single persisted timestamp
Precision Loss	Integer TTL truncation caused $\pm 1 ext{s}$ wobble	Exact epoch integer stored permanently
Network Execution	Separate non-atomic network round-trips	Single-hop atomic Lua script execution

> **Architectural Lesson:** Whenever distributed state relies on time, always persist absolute points in time rather than computing them dynamically from relative durations and multiple clocks.